Indian Institute of Technology, Kharagpur

Department of Electrical Engineering

Experiment 10 Report: Semantic Image Segmentation using UNet

Sandeep Saikia
Roll No: 24IE10003

Algorithms, AI and ML Laboratory (EE22202)
Spring Semester 2025–2026
Date: April 5, 2026

Contents

1 Introduction **2**

2 Dataset and Preparation **2**
2.1 Oxford-IIIT Pet Dataset . . . 2
2.2 Preprocessing and Splits . . . 2
2.3 Dataset Visualisation . . . 3

3 UNet Architecture **3**
3.1 Overall Design . . . 3
3.2 DoubleConv Block . . . 3
3.3 Skip Connections . . . 4
3.4 Trainable Parameter Count (Experiment 1 – Baseline) . . . 4

4 Loss Functions and Evaluation Metrics **5**
4.1 Pixel-wise Cross-Entropy Loss . . . 5
4.2 Dice Loss . . . 6
4.3 Combined Loss (Experiments 3 and 4) . . . 6
4.4 Evaluation Metrics . . . 6

5 Experiments **6**
5.1 Experiment 1 – Baseline UNet (Cross-Entropy Loss) . . . 6
5.1.1 (a) Trainable Parameters . . . 6
5.1.2 (b) Training and Validation Loss . . . 6
5.1.3 (c) Test Set Results . . . 7
5.2 Experiment 2 – Ablation: No Skip Connections . . . 7
5.2.1 (a) Trainable Parameters . . . 7
5.2.2 (b) Training and Validation Loss . . . 7
5.2.3 (c) Test Set Results . . . 8
5.3 Experiment 3 – Loss Ablation: Combined CE + Dice Loss . . . 8
5.3.1 (a) Trainable Parameters . . . 8
5.3.2 (b) Training and Validation Loss . . . 8
5.3.3 (c) Test Set Results . . . 8
5.4 Experiment 4 – Regularisation: Batch Normalisation and Dropout . . . 9
5.4.1 (a) Trainable Parameters . . . 9
5.4.2 (b) Training and Validation Loss . . . 9
5.4.3 (c) Test Set Results . . . 9

6 Comparison Across All Experiments **10**
6.1 Loss Curves . . . 10
6.2 Metrics Comparison . . . 10
6.3 Border Class Analysis . . . 11
6.4 Per-Class Dice Heatmap . . . 12
6.5 Qualitative Comparison . . . 13

7 Discussion and Key Observations **13**

8 Conclusion **14**

1 Introduction

In this experiment we implement and evaluate a UNet architecture for semantic image segmentation on the Oxford-IIIT Pet dataset. Unlike image classification where a single label is assigned to the whole image, segmentation requires labelling every pixel individually. This is called dense prediction and it demands a network that can preserve spatial resolution throughout processing while also understanding high-level context.

The UNet achieves this through a symmetric encoder-decoder structure. The encoder progressively reduces spatial dimensions to build up high-level feature representations, while the decoder gradually restores the original resolution. Crucially, skip connections link matching encoder and decoder layers so that fine-grained spatial detail is not lost during downsampling.

Four experiments are carried out in sequence to study the effect of skip connections, loss function choice, and regularisation techniques (batch normalisation and dropout) on segmentation quality. Performance is measured using mean Intersection over Union (mIoU) and the Dice coefficient on a held-out test set.

2 Dataset and Preparation

2.1 Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet dataset contains 7,349 RGB images covering 37 cat and dog breeds. Each image comes with a trimap annotation that assigns one of three classes to every pixel:

- **Class 1 – Pet:** Pixels belonging to the body of the animal.
- **Class 2 – Background:** All pixels outside the animal.
- **Class 3 – Border:** A thin boundary region between the pet and the background.

The border class occupies only a small fraction of pixels, so the dataset is class-imbalanced. This makes standard cross-entropy less suitable as a standalone loss because it treats all pixels equally regardless of class frequency.

2.2 Preprocessing and Splits

All images are resized to 256×256 pixels and normalised to $[-1, 1]$ using mean $(0.5, 0.5, 0.5)$ and standard deviation $(0.5, 0.5, 0.5)$. The mask values are originally 1, 2, 3 and are shifted to 0, 1, 2 for zero-indexed class labels. The dataset is split as follows:

Split	Samples
Training (80% of trainval)	2,944
Validation (20% of trainval)	736
Test (official test split)	3,669

Table 1: Dataset splits used across all experiments

2.3 Dataset Visualisation

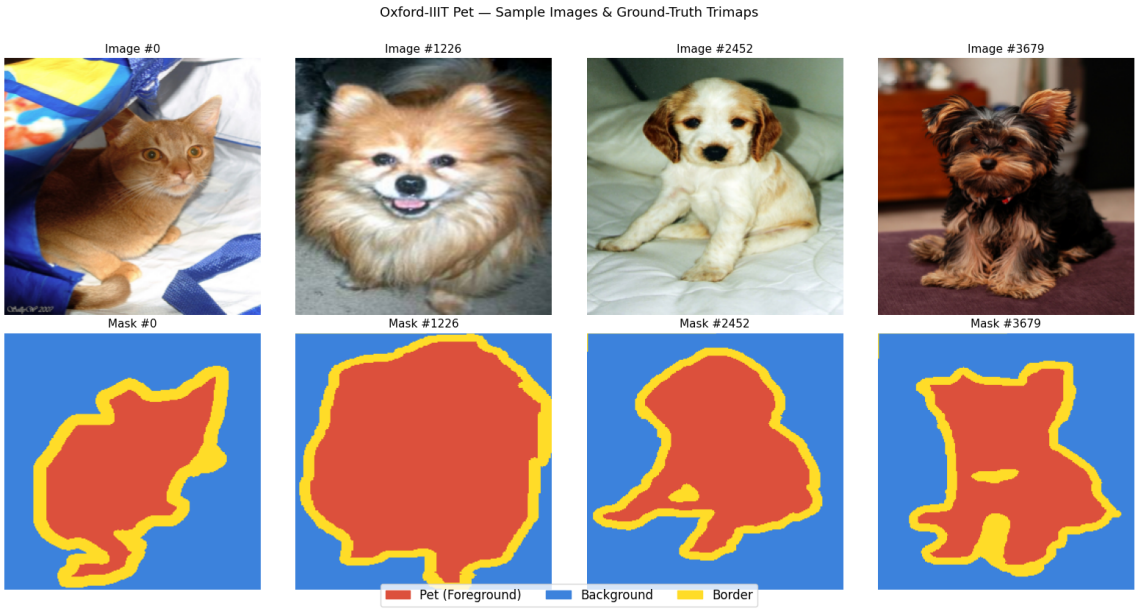

Figure 1: Sample images from the training set with their corresponding ground truth masks. Top row: RGB images. Bottom row: trimap masks (green = pet, grey = background, white = border).

3 UNet Architecture

3.1 Overall Design

The UNet consists of four encoder blocks, a bottleneck, and four decoder blocks. The encoder follows the contracting path where spatial dimensions are halved at each step and the number of channels is doubled. The decoder follows the expansive path where spatial dimensions are progressively restored. At each decoder step, an upsampled feature map is concatenated with the corresponding encoder feature map via a skip connection before being passed through a double convolution block. The final layer is a 1×1 convolution that maps the 64-channel feature map to 3 output classes (one logit per class per pixel).

3.2 DoubleConv Block

Each encoder and decoder block uses two consecutive 3×3 convolutions with padding 1 (to preserve spatial size within the block), each followed by ReLU. The block is configurable with optional batch normalisation and dropout:

Listing 1: DoubleConv block – the building unit of UNet

```
class DoubleConv(nn.Module):
    def __init__(self, in_ch, out_ch, use_bn=False, drop_p=0.0):
        super().__init__()
        bias = not use_bn
        layers = [
            nn.Conv2d(in_ch,  out_ch, kernel_size=3, padding=1,
               bias=bias),
```

```
            nn.BatchNorm2d(out_ch) if use_bn else nn.Identity(),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1,
                bias=bias),
            nn.BatchNorm2d(out_ch) if use_bn else nn.Identity(),
            nn.ReLU(inplace=True),
        ]
        if drop_p > 0:
            layers.append(nn.Dropout(drop_p))
        self.conv = nn.Sequential(*layers)
```

3.3 Skip Connections

At each decoder level, the upsampled feature map is concatenated with the feature map from the matching encoder level along the channel dimension:

$$\mathbf{f}_{\text{dec}}^{(l)} = \left[\mathbf{f}_{\text{up}}^{(l)},\ \mathbf{f}_{\text{enc}}^{(l)}\right]$$

where $[\cdot,\cdot]$ denotes channel-wise concatenation. This doubles the channel count before the DoubleConv block in the decoder, allowing the decoder to combine both semantic context and precise spatial boundary information.

3.4 Trainable Parameter Count (Experiment 1 – Baseline)

For a convolutional layer with C_{in} input channels, C_{out} output filters, kernel size $k \times k$, and bias:

$$\text{Params}_{\text{conv}} = (k \times k \times C_{\text{in}} + 1) \times C_{\text{out}}$$

For a transposed convolution (upsampling step) with kernel size $k = 2$:

$$\text{Params}_{\text{transpose}} = (k \times k \times C_{\text{in}}) \times C_{\text{out}} + C_{\text{out}}$$

Block	In → Out Channels	Spatial Size	Parameters
Enc Block 1 (DoubleConv)	$3 \to 64$	$256^2 \to 256^2$	38,720
MaxPool	–	$256^2 \to 128^2$	0
Enc Block 2 (DoubleConv)	$64 \to 128$	$128^2 \to 128^2$	221,440
MaxPool	–	$128^2 \to 64^2$	0
Enc Block 3 (DoubleConv)	$128 \to 256$	$64^2 \to 64^2$	885,248
MaxPool	–	$64^2 \to 32^2$	0
Enc Block 4 (DoubleConv)	$256 \to 512$	$32^2 \to 32^2$	3,540,480
MaxPool	–	$32^2 \to 16^2$	0
Bottleneck (DoubleConv)	$512 \to 1024$	$16^2 \to 16^2$	14,157,824
UpConv 1 (Transpose)	$1024 \to 512$	$16^2 \to 32^2$	2,097,664
Dec Block 1 (DoubleConv)	$1024 \to 512$	$32^2 \to 32^2$	7,079,424
UpConv 2 (Transpose)	$512 \to 256$	$32^2 \to 64^2$	524,544
Dec Block 2 (DoubleConv)	$512 \to 256$	$64^2 \to 64^2$	1,770,496
UpConv 3 (Transpose)	$256 \to 128$	$64^2 \to 128^2$	131,200
Dec Block 3 (DoubleConv)	$256 \to 128$	$128^2 \to 128^2$	443,008
UpConv 4 (Transpose)	$128 \to 64$	$128^2 \to 256^2$	32,832
Dec Block 4 (DoubleConv)	$128 \to 64$	$256^2 \to 256^2$	110,720
Final Conv (1×1)	$64 \to 3$	256^2	195
Total			**31,031,875**

Table 2: Experiment 1 layer-wise parameter count (PyTorch verified: 31,031,875 ✓)

4 Loss Functions and Evaluation Metrics

4.1 Pixel-wise Cross-Entropy Loss

The cross-entropy loss computes the classification error at each pixel independently across $C = 3$ classes:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{i,c} \log(\hat{y}_{i,c})$$

where N is the total number of pixels in the batch, $y_{i,c}$ is the one-hot ground truth label, and $\hat{y}_{i,c}$ is the predicted probability for class c at pixel i. This loss treats every pixel equally, so rare classes like the border region tend to be underrepresented in the gradient.

4.2 Dice Loss

The Dice loss directly measures the spatial overlap between the prediction and the ground truth:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum |Y \cap \hat{Y}|}{\sum |Y| + \sum |\hat{Y}| + \epsilon}$$

where ϵ is a small constant for numerical stability. Since it is computed over entire regions rather than pixel-by-pixel, it is much more sensitive to the minority border class and encourages the model to predict it more accurately.

4.3 Combined Loss (Experiments 3 and 4)

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{Dice}}$$

4.4 Evaluation Metrics

Mean Intersection over Union (mIoU): IoU for class c is $\frac{TP_c}{TP_c+FP_c+FN_c}$. The mean is taken over all three classes.

Dice Coefficient: $\text{Dice}_c = \frac{2 \cdot TP_c}{2 \cdot TP_c+FP_c+FN_c}$. The overall Dice is the macro-average over classes.

Training details fixed across all experiments: Adam optimiser, learning rate = 0.001, batch size = 16, maximum 30 epochs, early stopping with patience 7 (stops if validation loss does not improve for 7 consecutive epochs), learning rate scheduler (ReduceLROnPlateau, patience 3, factor 0.5).

5 Experiments

Exp	Skip Connections	Loss Function	Batch Norm	Dropout
1	Yes	Cross-Entropy	No	0.0
2	No	Cross-Entropy	No	0.0
3	Yes	CE + Dice	No	0.0
4	Yes	CE + Dice	Yes	0.3 (bottleneck)

Table 3: Summary of all four experiments. Everything else is kept identical.

5.1 Experiment 1 – Baseline UNet (Cross-Entropy Loss)

5.1.1 (a) Trainable Parameters

The baseline UNet has no batch normalisation or dropout. The full parameter breakdown is given in Table 2 above. Total trainable parameters: **31,031,875**.

5.1.2 (b) Training and Validation Loss

The model trained for 26 epochs before early stopping triggered. The best validation loss of 0.3239 was achieved at epoch 19. After that the validation loss began rising while training loss kept falling, indicating mild overfitting.

Epoch	Train Loss	Val Loss
5	0.4610	0.4302
10	0.3651	0.3705
15	0.2898	0.3266
19 (best)	0.2385	**0.3239**
26 (final)	0.1306	0.4339

Table 4: Experiment 1: loss at key epochs. Best validation loss is shown in bold.

5.1.3 (c) Test Set Results

Metric	Value
mIoU	0.6961
Dice	0.8076
Per-class IoU – Pet	0.7621
Per-class IoU – Background	0.8683
Per-class IoU – Border	0.4580

Table 5: Experiment 1 test set results

The model segments the pet body and background reasonably well. The border class IoU of 0.4580 is noticeably lower, reflecting the challenge of predicting a thin and visually ambiguous boundary region using a loss that does not specifically target minority classes.

5.2 Experiment 2 – Ablation: No Skip Connections

5.2.1 (a) Trainable Parameters

Removing skip connections means the decoder DoubleConv blocks no longer receive the concatenated encoder features. Each decoder DoubleConv now takes half the input channels it did in Experiment 1 (e.g., 512 instead of 1024 for Dec Block 1). This reduces the total parameter count to **27,898,435**.

5.2.2 (b) Training and Validation Loss

Early stopping triggered at epoch 20. The model converged faster but to a worse validation loss, suggesting that without skip connections the decoder is learning an inferior approximation of the segmentation boundaries.

Epoch	Train Loss	Val Loss
5	0.5005	0.4949
10	0.3734	0.3943
13 (best)	0.3092	**0.3706**
20 (final)	0.1449	0.4726

Table 6: Experiment 2: loss at key epochs.

5.2.3 (c) Test Set Results

Metric	Value
mIoU	0.6595
Dice	0.7778
Per-class IoU – Pet	0.7284
Per-class IoU – Background	0.8501
Per-class IoU – Border	0.4000

Table 7: Experiment 2 test set results

Removing skip connections caused mIoU to drop by 3.66 percentage points and the border IoU to drop by 5.8 points relative to the baseline. The decoder can still reconstruct a rough segmentation from the bottleneck alone, but it loses the fine spatial detail needed to accurately predict object boundaries.

5.3 Experiment 3 – Loss Ablation: Combined CE + Dice Loss

5.3.1 (a) Trainable Parameters

The architecture is identical to Experiment 1. Only the loss function changes. Total trainable parameters: **31,031,875**.

5.3.2 (b) Training and Validation Loss

With the combined loss, the raw loss values are larger because $\mathcal{L}_{\text{CE}}$ and $\mathcal{L}_{\text{Dice}}$ are summed directly. Early stopping triggered at epoch 22. The best validation loss of 0.5419 was reached at epoch 15.

Epoch	Train Loss	Val Loss
5	0.7858	0.7570
10	0.6061	0.5816
15 (best)	0.4691	**0.5419**
20	0.3167	0.5544
22 (final)	0.2692	0.6017

Table 8: Experiment 3: loss at key epochs. Note higher absolute values due to summed CE and Dice losses.

5.3.3 (c) Test Set Results

Metric	Value
mIoU	0.7198
Dice	0.8254
Per-class IoU – Pet	0.7864
Per-class IoU – Background	0.8823
Per-class IoU – Border	0.4906

Table 9: Experiment 3 test set results

Adding the Dice loss improved mIoU from 0.6961 to 0.7198 and boosted border class IoU from 0.4580 to 0.4906, confirming that the Dice component specifically helps with the minority class. This makes Experiment 3 the best model so far and the starting point for Experiment 4.

5.4 Experiment 4 – Regularisation: Batch Normalisation and Dropout

5.4.1 (a) Trainable Parameters

Batch normalisation (BN) is added inside every DoubleConv block. Each BN layer introduces $2C$ learnable parameters (scale γ and shift β per channel). Dropout with $p = 0.3$ is applied in the 1024-channel bottleneck layer only. Dropout introduces no learnable parameters.

The extra BN parameters across all encoder and decoder DoubleConv blocks add up to 5,888 additional parameters ($= 31,037,763 - 31,031,875$). Total trainable parameters: **31,037,763**.

5.4.2 (b) Training and Validation Loss

This is the only experiment that ran the full 30 epochs without early stopping. The combined BN and dropout regularisation kept validation loss from diverging even as training loss continued to decrease. The best validation loss of 0.4665 was at epoch 30.

Epoch	Train Loss	Val Loss
5	0.8086	0.8120
10	0.6388	0.6393
15	0.5539	0.5516
20	0.4712	0.5621
25	0.3962	0.4885
30 (final)	0.3212	**0.4665**

Table 10: Experiment 4: loss at key epochs.

5.4.3 (c) Test Set Results

Metric	Value
mIoU	0.7516
Dice	0.8483
Per-class IoU – Pet	0.8193
Per-class IoU – Background	0.9014
Per-class IoU – Border	0.5340

Table 11: Experiment 4 test set results

This is the best performing model across all four experiments. The combination of CE + Dice loss with BN and bottleneck dropout pushed mIoU to 0.7516 and border class IoU to 0.5340, a gain of 7.6 mIoU points over the baseline.

6 Comparison Across All Experiments

6.1 Loss Curves

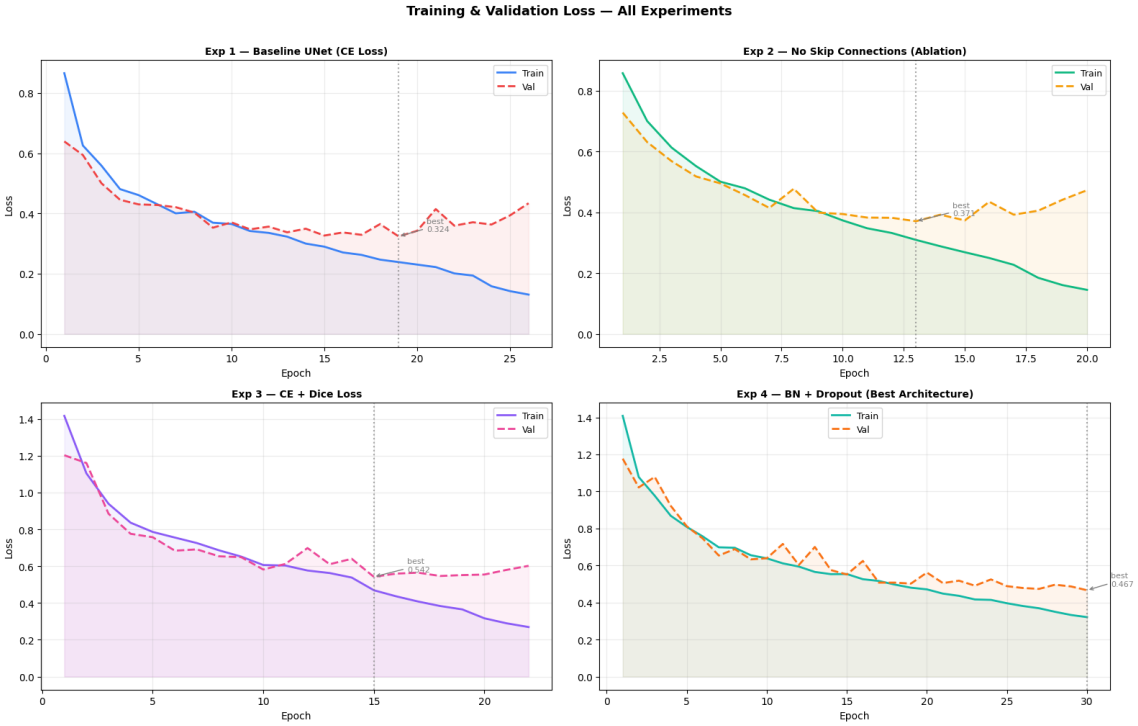

Figure 2: Training (solid) and validation (dashed) loss curves for all four experiments. Each subplot corresponds to one experiment. Vertical axes are on the same scale within each subplot. The shaded region marks the gap between training and validation loss. Experiments 1 and 2 show earlier overfitting; Experiment 4 sustains close training and validation loss for all 30 epochs.

6.2 Metrics Comparison

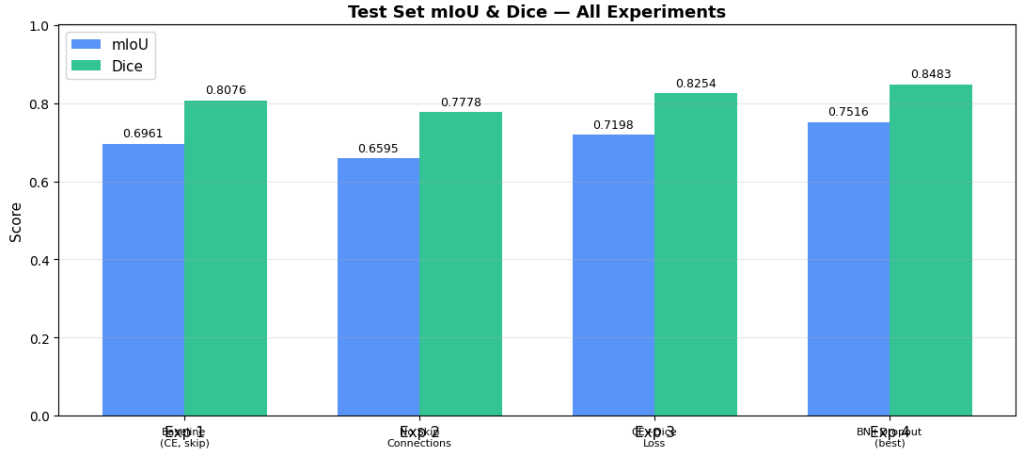

Figure 3: mIoU and Dice coefficient on the test set for all four experiments. Higher is better for both metrics. Experiment 4 achieves the highest scores on both.

Experiment	Params	mIoU	Dice	Key Change
Exp 1: Baseline (CE)	31,031,875	0.6961	0.8076	Baseline
Exp 2: No Skip Conn.	27,898,435	0.6595	0.7778	Remove skip connections
Exp 3: CE + Dice	31,031,875	0.7198	0.8254	Add Dice loss
Exp 4: BN + Dropout	31,037,763	**0.7516**	**0.8483**	Add BN + bottleneck dropout

Table 12: Summary of all four experiments. Bold indicates the best result.

6.3 Border Class Analysis

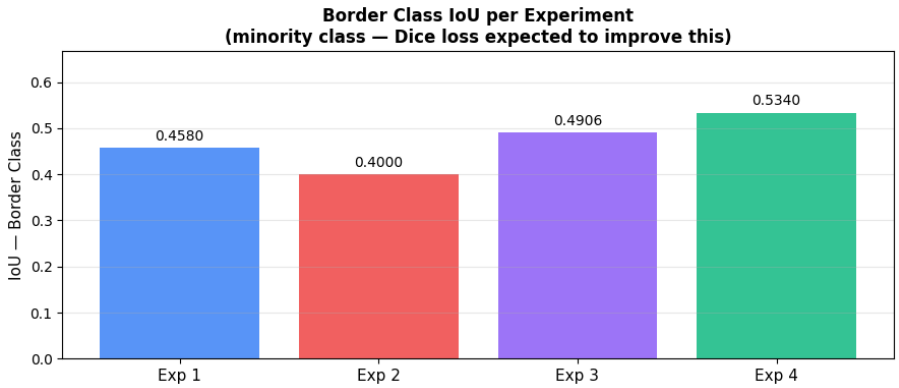

Figure 4: IoU for the minority border class across all four experiments. The jump from Exp 2 to Exp 3 is due to adding the Dice loss, which directly penalises missed border pixels. Exp 4 achieves the highest border IoU of 0.5340.

The border class is the hardest to segment correctly. With plain cross-entropy (Exp 1 and Exp 2) the border IoU stays below 0.46. Once the Dice loss is added (Exp 3 and Exp 4), the model learns to pay more attention to these boundary pixels and border IoU rises meaningfully.

6.4 Per-Class Dice Heatmap

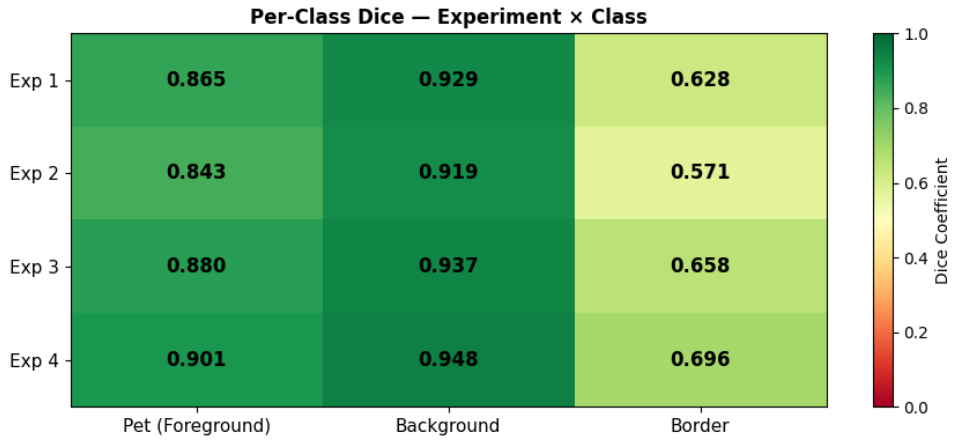

Figure 5: Per-class Dice coefficients for each experiment visualised as a heatmap (green = high, red = low). The border class (rightmost column) consistently has the lowest Dice, confirming it is the most difficult class. Experiment 4 shows the most uniform green pattern across all classes.

6.5 Qualitative Comparison

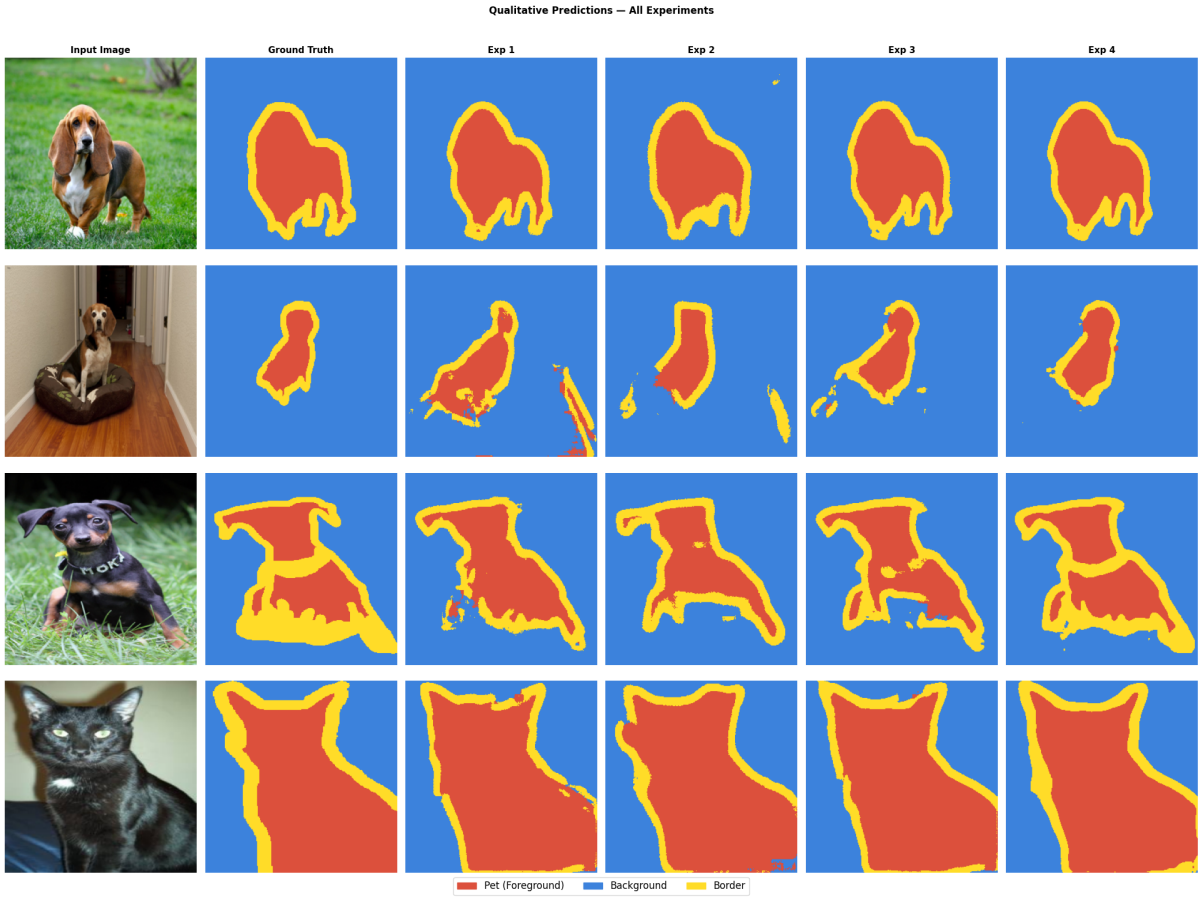

Figure 6: Qualitative segmentation results on four test images. Columns left to right: input image, ground truth mask, Exp 1 prediction, Exp 2 prediction, Exp 3 prediction, Exp 4 prediction. Colour coding: green = pet, grey = background, white/yellow = border.

Looking at the qualitative results, a few patterns stand out. Experiment 2 (no skip connections) produces noticeably blurrier boundaries – the border region is often merged into either pet or background. Experiments 3 and 4 generate cleaner and more precise boundary predictions that align more closely with the ground truth. In particular, Experiment 4 recovers fine details like the edges of ears and tails that are missed by the baseline.

7 Discussion and Key Observations

Skip connections are essential for boundary precision. Removing them (Exp 2) cost 3.7 mIoU points and 3 Dice points relative to the baseline. The encoder feature maps contain low-level spatial information that the bottleneck representation alone cannot reconstruct – this confirms the core design motivation of UNet.

The Dice loss is the biggest single improvement. Going from Exp 1 to Exp 3 (same architecture, just adding Dice to the loss) improved mIoU by 2.4 points and border class IoU by 3.3 points. The cross-entropy loss has no mechanism to compensate for class

imbalance, so the minority border class is systematically under-predicted. The Dice loss fixes this by directly optimising spatial overlap.

Batch normalisation and dropout provide additional stability. Experiment 4 is the only model to run all 30 epochs without early stopping. The training and validation losses stayed much closer together throughout, whereas in Experiments 1–3 the validation loss began rising well before training ended. BN stabilises the gradient flow through the deep network and dropout prevents the bottleneck from memorising specific training patterns.

Class difficulty ordering is consistent. Across all four experiments, the per-class IoU follows the same order: Background > Pet > Border. The background is the easiest because it occupies the most pixels and has the most uniform appearance. The border is always the hardest because it is thin, visually ambiguous, and heavily outnumbered by the other two classes.

More parameters does not mean better results. Experiment 2 has fewer parameters than all others (27.9M vs 31.0M) yet performs the worst. This rules out parameter count as the explanation for performance differences and points instead to architecture and loss function design choices.

8 Conclusion

In this experiment, we trained and evaluated four UNet configurations for semantic segmentation of the Oxford-IIIT Pet dataset. The key findings are:

- The baseline UNet with skip connections and cross-entropy loss achieves a test mIoU of 0.6961 and a Dice of 0.8076.
- Removing skip connections (Exp 2) causes a clear drop in mIoU to 0.6595, particularly hurting boundary localisation and the border class IoU.
- Switching to a combined CE + Dice loss (Exp 3) improves mIoU to 0.7198 and significantly boosts border class prediction, confirming Dice loss as the right choice for imbalanced segmentation tasks.
- Adding batch normalisation in all DoubleConv blocks and dropout at the bottleneck (Exp 4) further improves mIoU to 0.7516 and Dice to 0.8483, with the best per-class scores across all three classes.

Overall, the combination of skip connections, a mixed CE + Dice loss, and regularisation via batch normalisation and dropout gives the best results. Each of these design choices addresses a distinct limitation of the simpler models, and their effects stack together to produce a measurably better segmentation system.